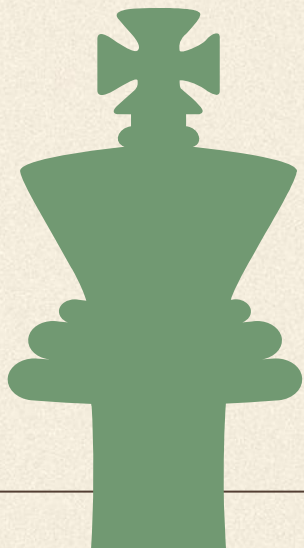


AI 체스



협동2 A-2(영남향우회)

조장: 박윤현

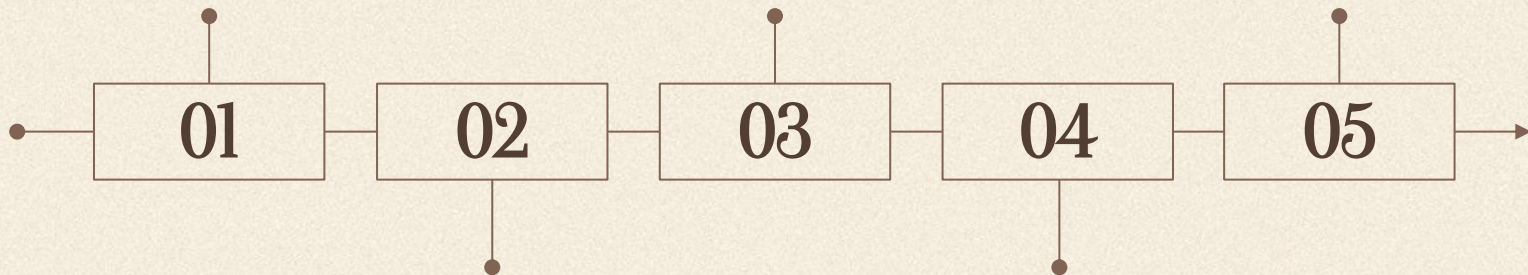
팀원: 김연빈 김하균 문규혁 서재우

목차

프로젝트 개요

수행 절차 & 방법

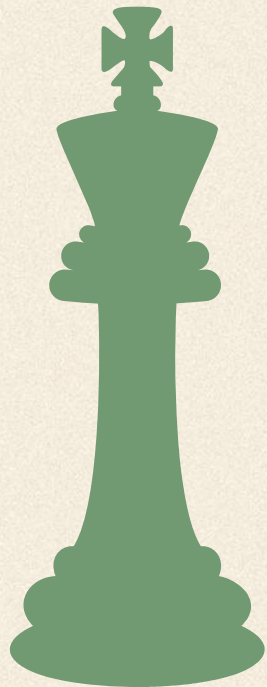
자체평가 & 의견



팀 구성 & 역할

수행 경과

개요



왜 체스인가?

- 높이 5cm, 너비 3cm 내외의 비교적 작고, 2개의 색과 6개의 모양을 가지는 세트
- 객체 인식 및 음성 인식 ➡ 로봇 동작으로 이어지는 하나의 시퀀스를 적절하게 제시 가능하다고 판단
- 객체 인식에 사용한 기법, 로봇 동작의 정밀도를 다른 분야로 확장하기에도 용이하다는 판단

기술 스택

전체 시스템

ROS 2



Robot Control

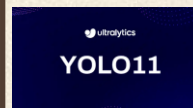


Doosan Robotics
두산로보틱스

Cloud & UI



Data Analysis & Vision



팀 구성 & 역할

- I. 전체 일정
- II. 개인 별 일정

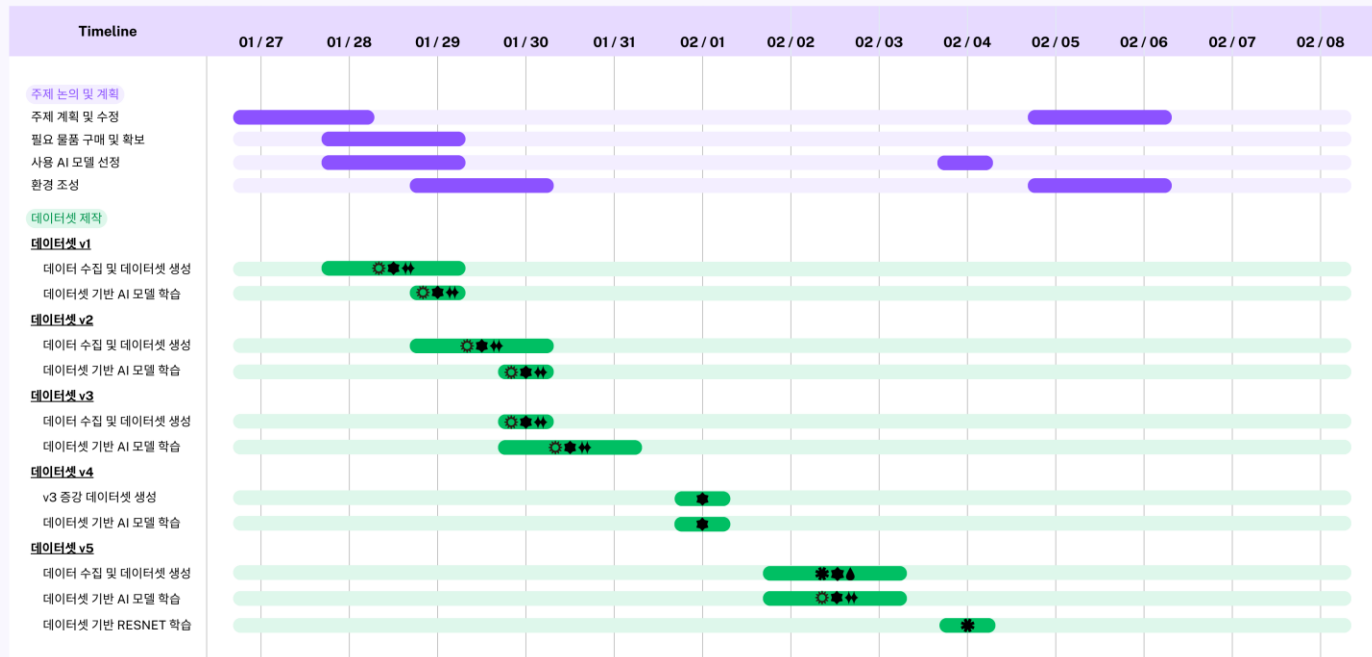


전체 일정

협동2 A-2 AI 체스 타임라인

주: AI 학습

주: 기능 노드 코딩



협동2 A-2 AI 체스 타임라인

주: 기능 노드 코딩



한글



문규호



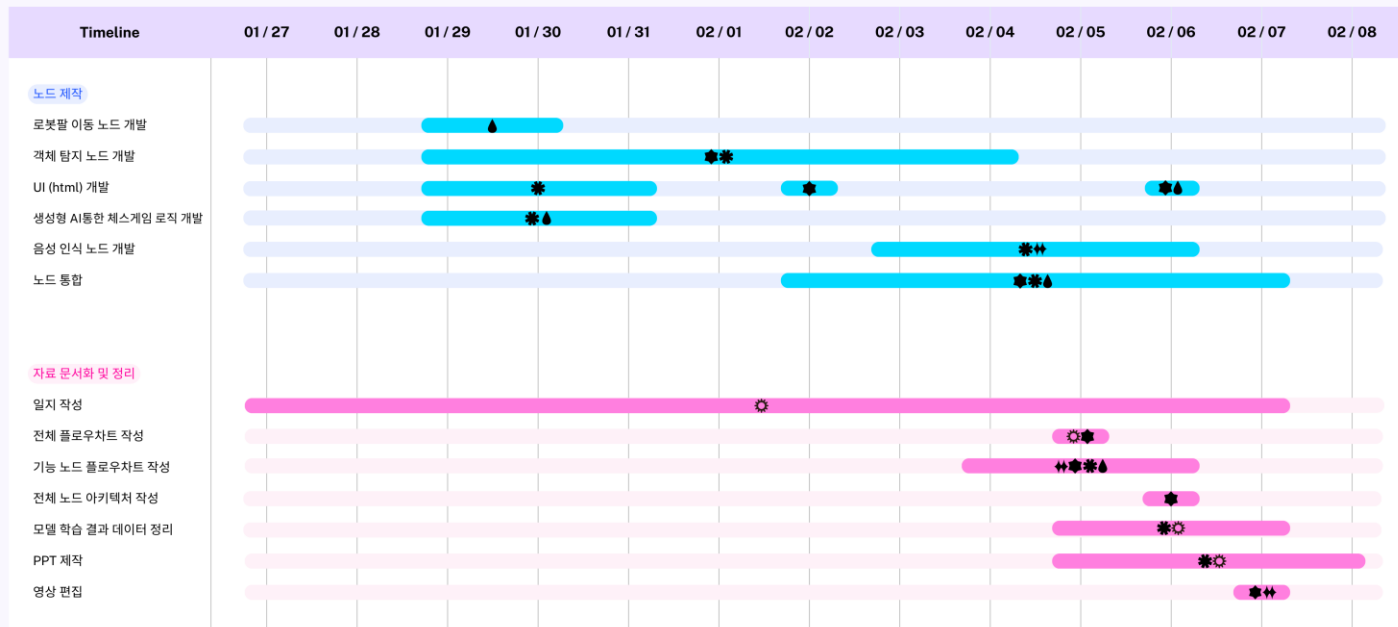
김여빈



서재우 김



김하균



개인 별 일정

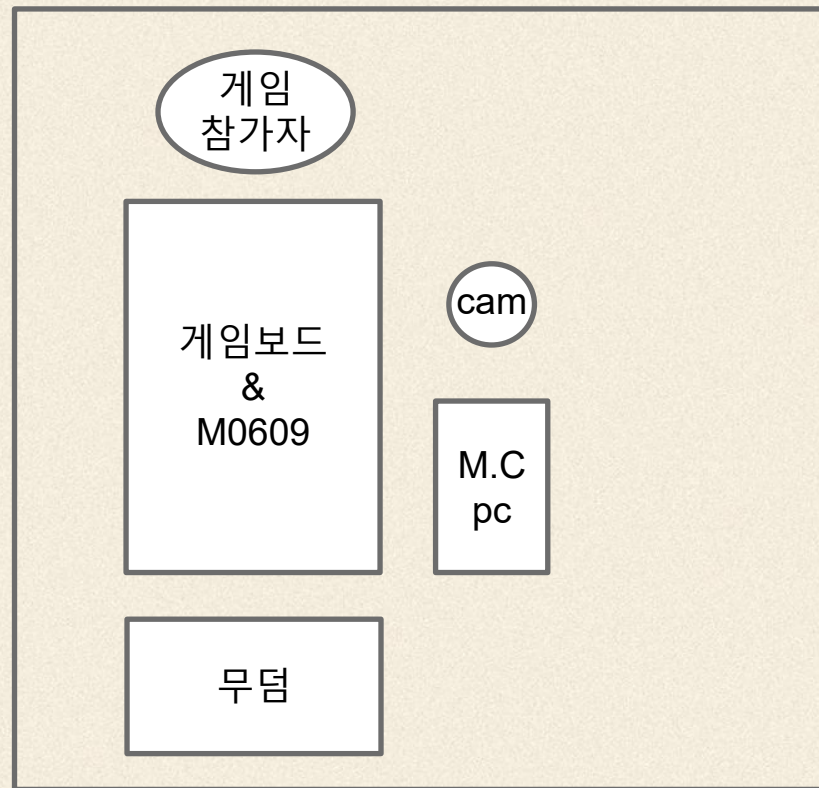
Team Member	Phase 1 (1.27 ~ 1.29)	Phase 2 (1.30 ~ 2.1)	Phase 3 (2.2 ~ 2.4)	Phase 4 (2.4 ~ 2.9)
박윤현	- 데이터 라벨링 - YOLOv8n 학습	- YOLOv8n 학습 - 데이터셋 v3/v4 구축	- YOLOv8n 모델 평가 - 데이터셋 v5 정리	자료 정리 및 문서화
김하균	- 로봇팔 이동 노드 작성 - ROS2 기본 설정	- Node Architecture 설계 - 시스템 구조 최적화	- MobileNet 노드 구현 - 비전-로봇 연동 테스트	- 전체 코드 통합 - 최종 시연 디버깅
김연빈	- 데이터 라벨링 - YOLOv11/26 학습	- 데이터셋 v3 기반 학습 - 데이터셋 v4 확장/학습	- 모델 최종 평가 - 데이터셋 v5 확정	전체 시스템 통합
문규혁	- 데이터 라벨링 - YOLOv8s 학습	- YOLOv8s 학습 - 음성 데이터 수집	- YOLOv8s 모델 평가 - 음성 인식 노드 제작	- 음성 노드 최종 통합 - 시연 시나리오 점검
서재우	- Web UI 기획 및 생성 - 초기 프론트엔드 구축	- 생성형 AI 체스 로직 구현 - LLM 연동 테스트	- YOLOv11 추가 학습 - 음성 노드 제작 지원	- 비전 코드 최종 통합 - 인식 성능 최적화

수행 절차 & 방법

- I. 개발 환경
- II. Flow chart
- III. DataSet
- IV. Object Detection
- V. UI/ Firebase DB log



게임 공간



- 음성 명령 대기
- 환경 설정 로드

- 보드 상태 샘플링
- AI 수 계산
- 사용자 검증

- 로봇 제어
- 상태 업데이트
- 대기 복귀



DataSet



Image samples from ChessReD

End-to-End Chess Recognition git에서
10800장 이미지 data sample 사용



Chess Image + Augmentation Image

직접 찍은 Image data sample 1600장과
해당 이미지를 Augmentation 적용한
1600장 사용

DataSet



80%

Train

모델의 가중치 학습을 위한
핵심 데이터셋



10%

Val

학습 과정 중 실시간 성능
검증



10%

Test

학습에 전혀 관여하지 않은
데이터

- Train ➡ **11,200** Images, 다양한 조명, 각도, 배경 노이즈를 포함하여 일반화 성능 확보
- Val ➡ **1,400** Images, 과적합 방지 및 하이퍼파라미터 튜닝 지표로 활용
- Test ➡ **1,400** Images, 최종 모델의 실제 환경 인식을 및 신뢰도 평가

모델 비교(OpenAI)



결과 반환
(텍스트/ JSON)

장점

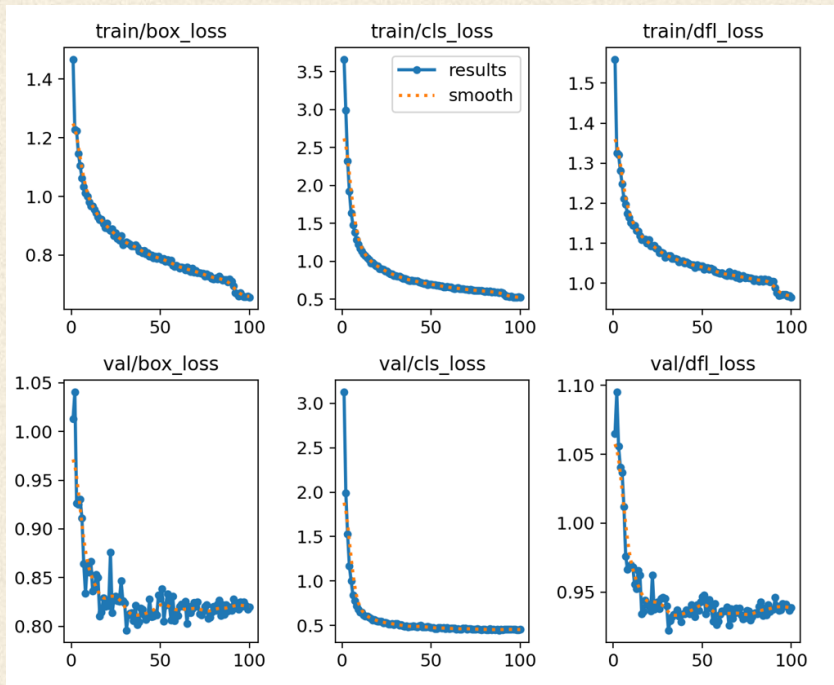
- 모델 학습 불필요
- 강력한 사전 학습 성능 활용

단점

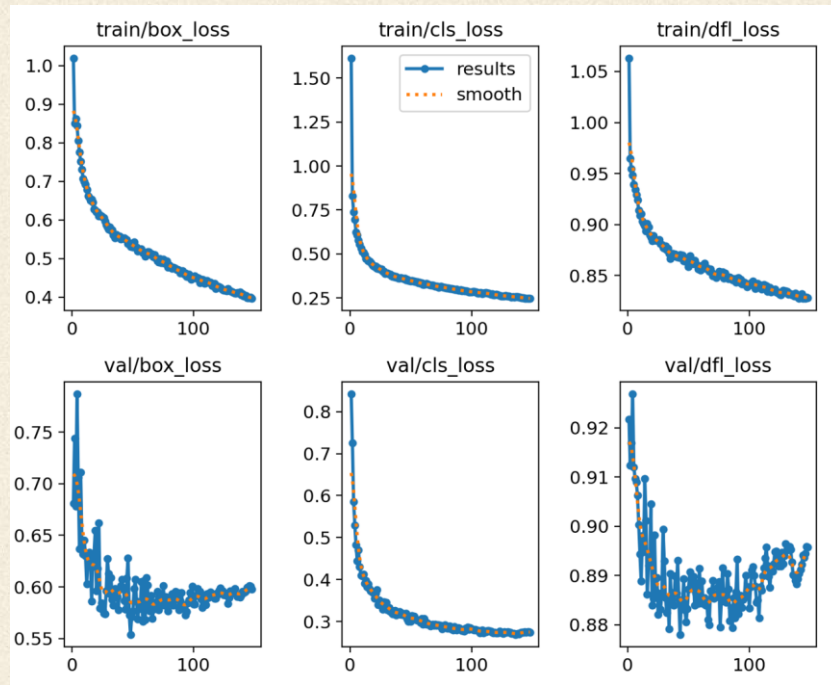
- API 호출 비용 발생
- 네트워크 지연 시간
- 미세한 기물 구분 정확도 구조(LLM은 텍스트에 특화)

모델 비교(YOLO v8)

Epoch = 100
Batch size = 16
Image size = 1080

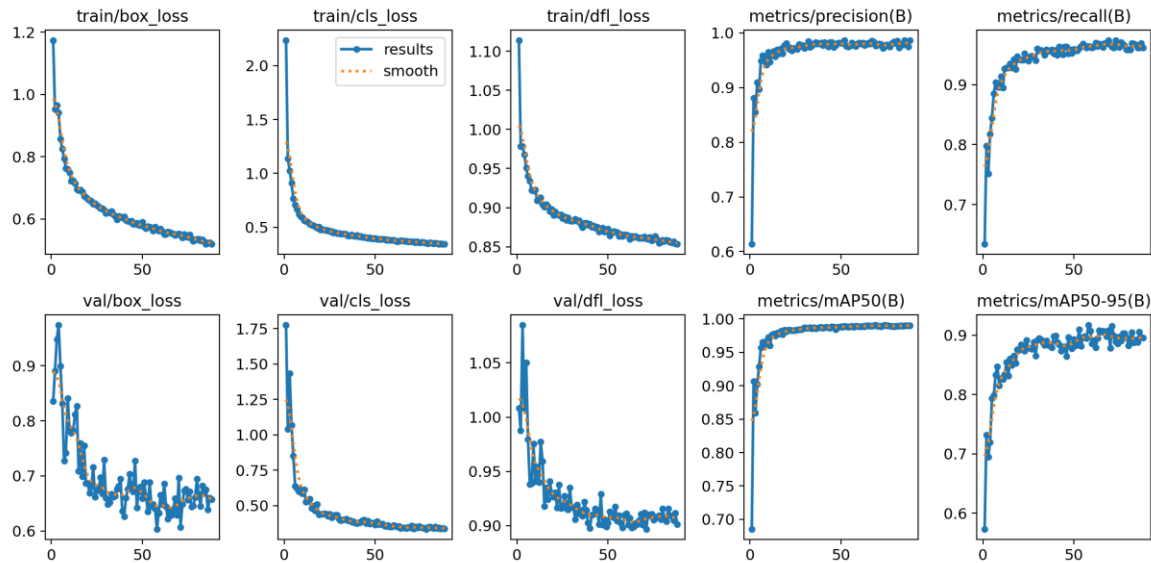


YOLO v8n



YOLO v8s

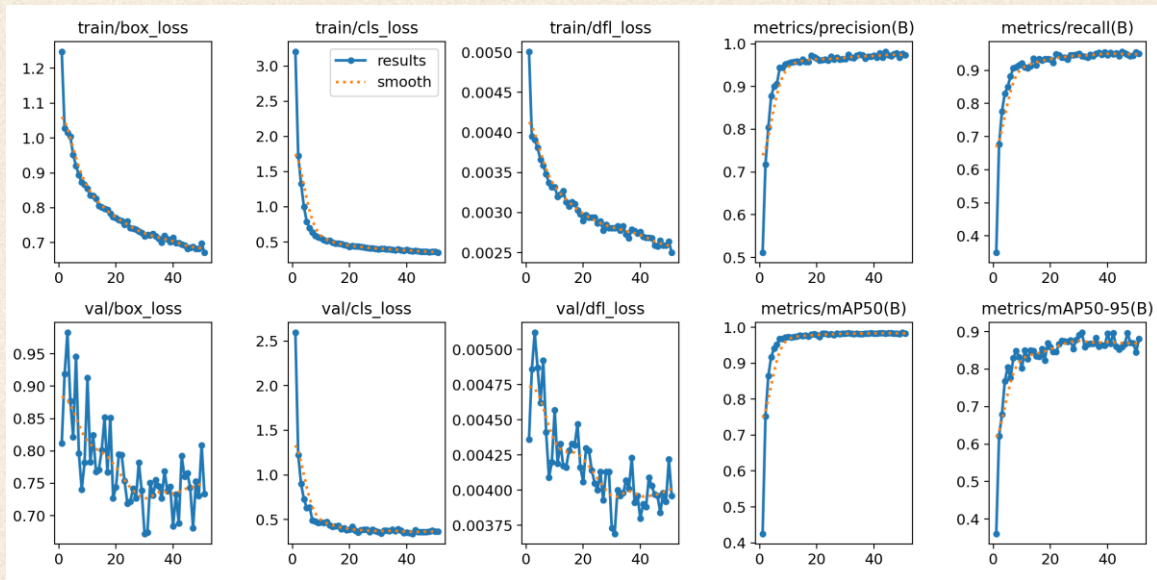
모델 비교(YOLO v11n)



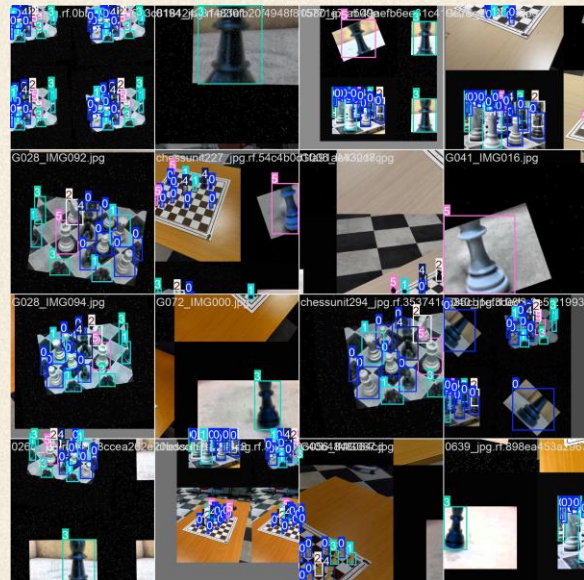
YOLO v11

Confidence

모델 비교(YOLO v26)



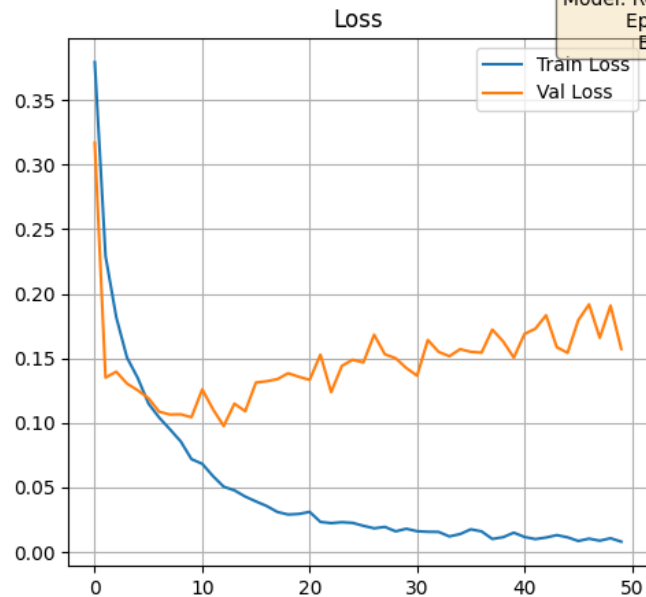
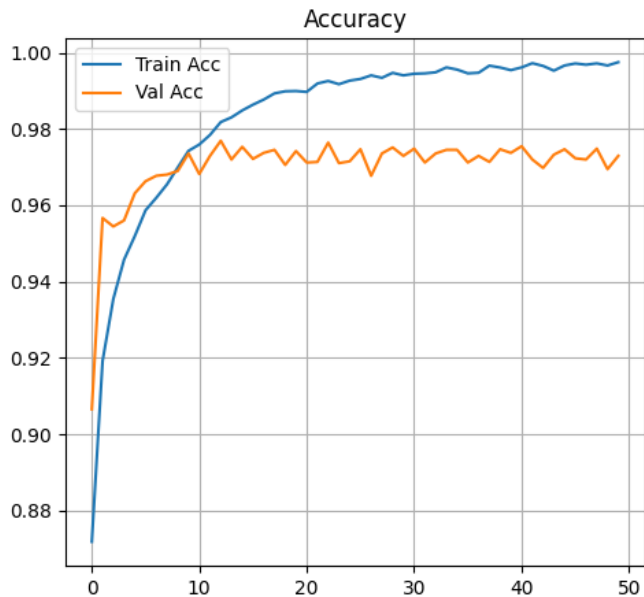
YOLO v26



Confidence

모델 비교(ResNet 18)

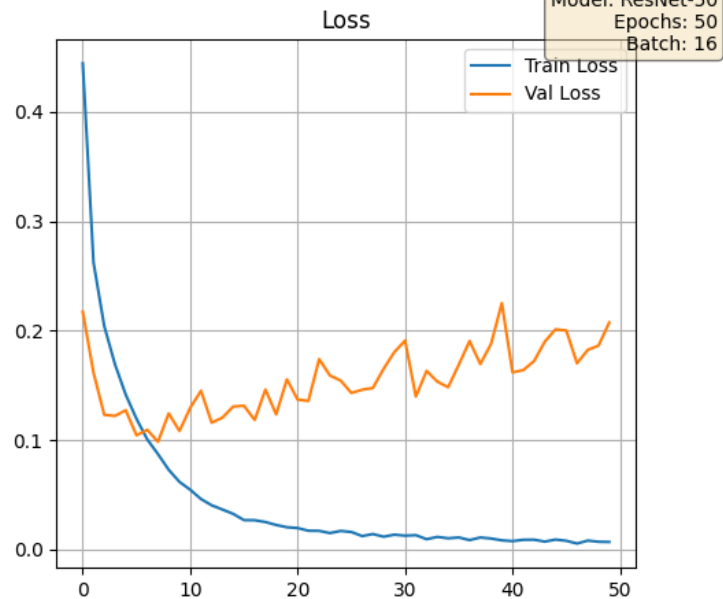
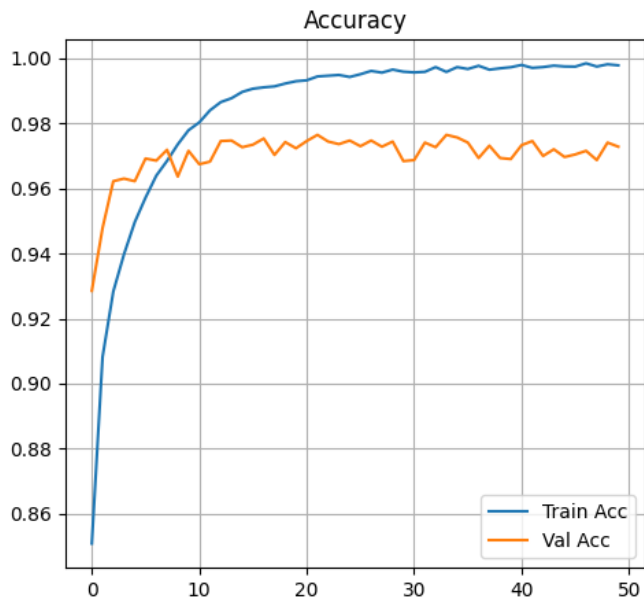
classification-ResNet-18

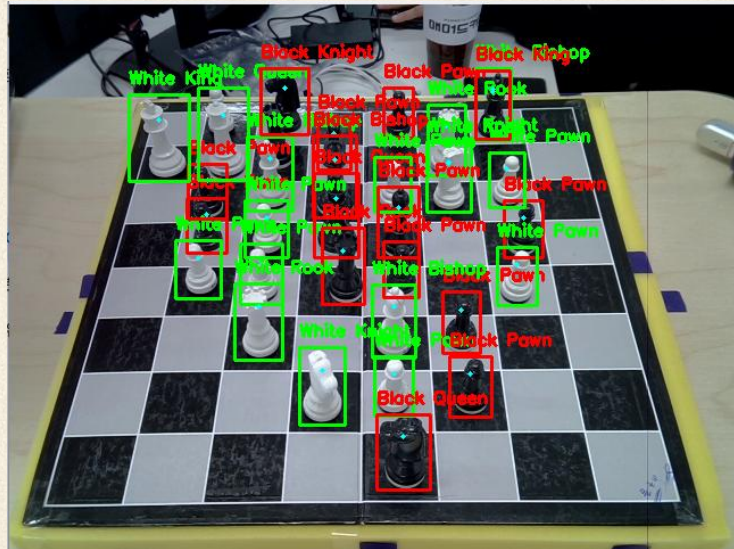


Model: ResNet-18
Epochs: 50
Batch: 64

모델 비교(ResNet 50)

classification-ResNet-50





- 네 모서리의 점을 토대로 원근 변환 행렬을 만들고 64분할하여 그리드 생성한다. 해당 bbox 아랫변이 절반 이상 점유한 칸을 좌표로 인식
- 바닥과 구분하기 위하여 기물만 Crop 적용하고, 해당 기물에 픽셀 값을 통하여 흑백 판별

최종 선정 모델

	모델 명	YOLO 11n
	역할	전체 보드 상에서 기물의 전체적인 위치 파악, bbbox 생성

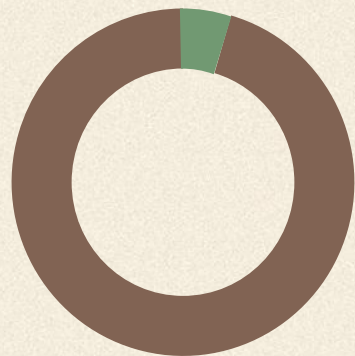


	모델 명	ResNet18, OpenCV
	역할	bbbox 내 기물의 class 파악 - 흑/백 구분 및 좌표 저장

Object Detection

특수한 상황(기물이 카메라와 밀접 또는 반사되는 빛)
이외에는 오차가 발생하지 않는 객체 인식 모델 설계

95%



UI/ Firebase DB log



UI

UI/ Firebase DB log

I. Board_state(Vision -> DB)

- YOLO + ResNet이 인식한 실시간
보드 좌표

II. Chess_system(Main -> DB)

- 게임 난이도(Difficulty), 턴(Turn),
시스템 상태 관리

III. Ui_control(User <-> Robot)

- AI 추천 수, 사용자 승인, 검증 플래그

❖ 시스템 간 비동기 통신을 위한 공유 메모리 역할



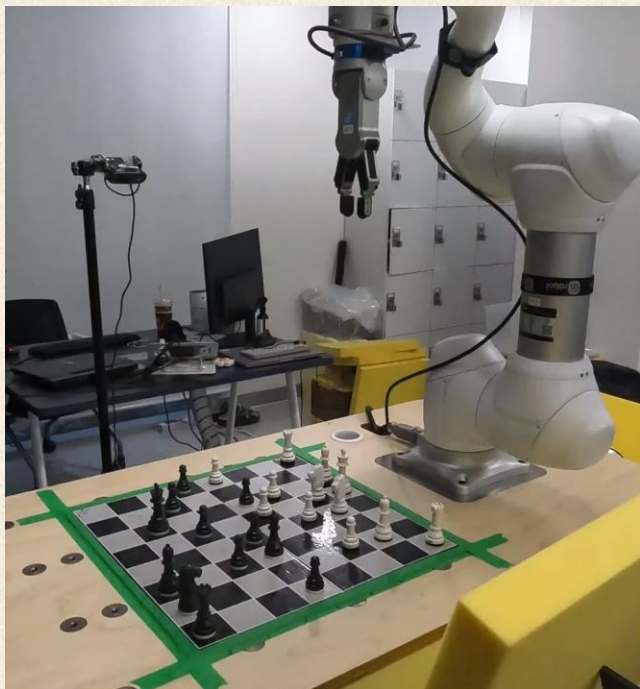
```
GD https://chess-43355-default-rtdb.asia-southeast1.firebaseio.com/.
https://chess-43355-default-rtdb.asia-southeast1.firebaseio.com/.
├── chess
│   ├── board_state
│   │   ├── board
│   │   │   ├── A5: "BP"
│   │   │   ├── B4: "WP"
│   │   │   ├── B6: "BP"
│   │   │   ├── C5: "BN"
│   │   │   ├── C7: "BQ"
│   │   │   ├── D3: "WR"
│   │   │   ├── D8: "BO"
│   │   │   ├── E1: "WK"
│   │   │   ├── E5: "WQ"
│   │   │   ├── E6: "WP"
│   │   │   ├── F6: "WP"
│   │   │   ├── H4: "BP"
│   │   └── piece_count: 12
│   └── updated_at: "2026-02-06T18:30:09.619"
├── chess_system
│   ├── depth: 15
│   ├── difficulty: 10
│   ├── turn: "w"
│   └── updated_at: "2026-02-06T09:04:47.145Z"
└── ui_control
    ├── ai_suggested_move: "fed7"
    ├── ai_updated_at: "2026-02-06T18:12:14.046526"
    ├── command: "idle"
    ├── status: "sampling"
    ├── final_board
    ├── job_id: "2026-02-06T18:13:46.425"
    ├── timestamp: "2026-02-06T18:13:59.222207"
    ├── user_decision: "NONE"
    ├── verification: false
    ├── voice_message: "Hello rokey로 불러주세요"
    ├── voice_updated_at: "2026-02-06T18:14:03.317591"
    └── working: false
```


수행 경과

- I. 최종 시연 영상
- II. Node architecture



최종 시연 영상



Node architecture

I. 핵심 컨셉

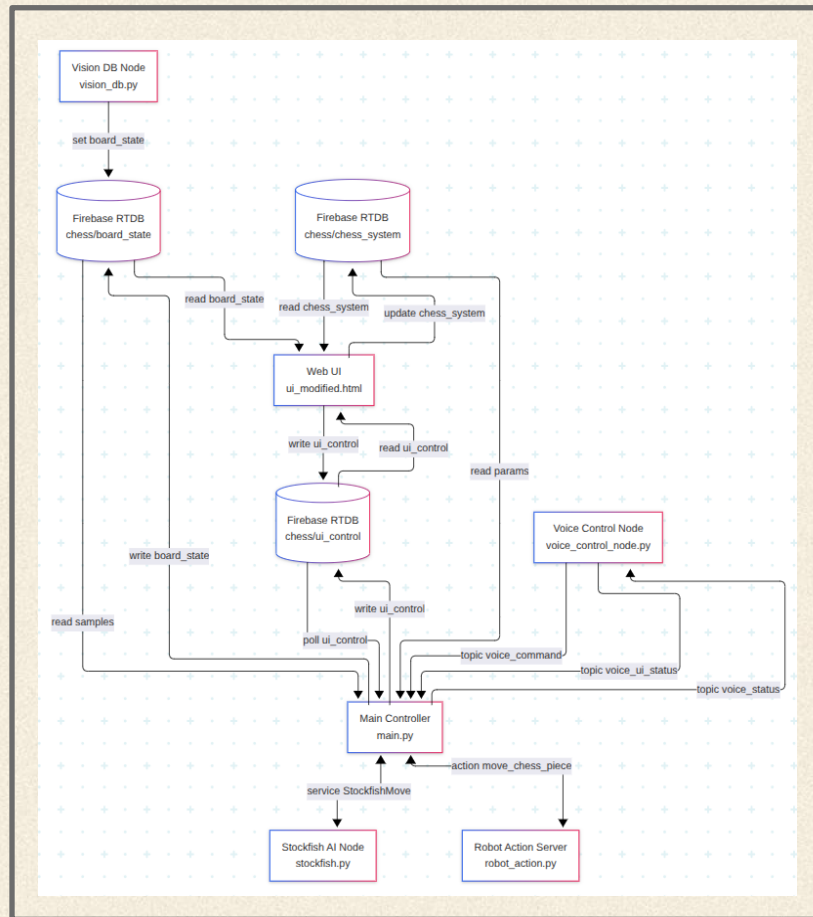
- 이기종 통신 통합
- 중앙 집중식 제어

II. 구성 요소별 역할

- Vision & Cloud
- Intelligence
- Action & Voice

III. 데이터 흐름의 특징

- 비동기
- 신뢰성
- 상호작용



자체 평가 의견

Project Pros	
하이브리드 비전 파이프라인의 높은 정확도	단순히 YOLO 하나만 사용하는 것이 아니라, YOLOv11로 위치를 검출하고 ResNet-18로 정밀 분류를 수행하는 2단계 방식을 채택했습니다.
Firebase를 통한 시스템 유연성 및 확장성	서로 다른 환경을 Firebase Realtime Database를 매개체로 연결하였고, 이 방식은 각 시스템이 독립적으로 작동하면서 데이터를 공유하므로, 특정 파트의 부하가 전체 시스템을 멈추게 하지 않는 '디커플링(Decoupling)' 효과를 제공
Human-in-the-Loop 설계를 통한 안전성 확보	수를 계산하고 바로 두지 않고 사용자의 승인을 받는 단계를 두어 경기 오류를 완벽히 차단
다수결 알고리즘 기반의 데이터 정제	비전 데이터를 한 번만 보고 판단하지 않고 5회 이상 샘플링하여 가장 빈도가 높은 상태를 채택
직관적인 멀티모달 인터페이스	음성인식과 터치 기반 웹 UI를 동시에 지원 기술적 장벽이 낮은 일반 사용자도 손쉽게 사용

Project Cons	
클라우드 의존성으로 인한 지연 시간 (Latency)	로컬 통신이 아닌 Firebase 클라우드를 거치기 때문에 네트워크 상태에 따라 반응 속도가 달라질 수 있으며, 실시간성이 극도로 중요한 작업에서는 0.1~0.5초의 지연이 누적되어 사용자 경험에 답답함을 느낄 수 있음
물리적 환경 변화에 대한 민감도	비전 시스템에서 OpenCV를 활용해 좌표를 매핑하므로, 카메라의 위치나 체스판의 각도가 미세하게 틀어지면 로봇의 좌표 계산에 오류 발생 여지 존재
복잡한 시스템 기동 프로세스	시스템을 가동하기 위해 데이터를 실시간으로 스트리밍을 해야하는 복잡함이 있음.

I. Appendix

- **C3k2 블록 도입 (Backbone/Neck/Head 전반 적용)**

기존 YOLOv8의 C2f 블록 대신, 더 가볍고 빠른 **C3k2** 블록을 사용하고, 작은 커널 사이즈(2)를 이용해 연산량을 줄이면서도 특징 추출 성능을 유지함.

덕분에 속도는 빨라지고, 모델 크기는 작아짐

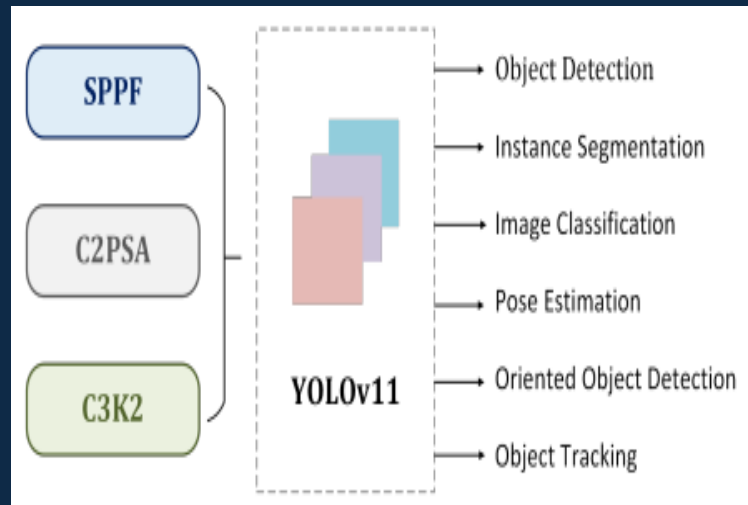
- **C2PSA 블록 추가 (Parallel Spatial Attention)**

SPPF 블록 다음에 **C2PSA**(병렬 공간 주의 메커니즘)을 배치 모델이 이미지 내 '중요한 부분'에 더욱 집중할 수 있게 되어, 작은 객체나 가려진 객체 검출 성능이 향상

- **다양한 사이즈의 모델 제공**

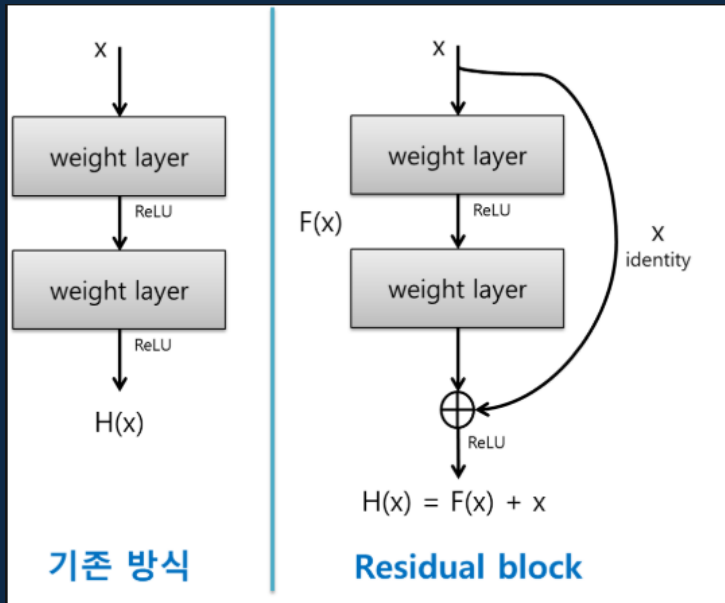
Nano, Small, Medium, Large, X-Large 버전이 존재

경량 엣지 디바이스부터 고성능 서버까지 다양한 환경에 최적화 가능



II. Appendix

- 기존의 신경망은 입력값 x 를 타겟값 y 로 매핑하는 함수 $H(x)$ 를 얻는 것이 목적이지만, **ResNet은 $F(x) + x$ 를 최소화하는 것을 목적으로 한다.** x 는 현시점에서 변할 수 없는 값이므로 $F(x)$ 를 0에 가깝게 만들어, $F(x)$ 가 0이 되면 출력과 입력이 모두 x 로 같아지게 된다. **$F(x) = H(x) - x$** 이므로 $F(x)$ 를 최소화 해준다는 것은 $H(x) - x$ 를 최소화 해주는 것과 동일한 의미를 지닌다. 여기서 $H(x) - x$ 를 **잔차(residual)**라고 한다. 즉, 잔차를 최소화 해주는 것이므로 ResNet이란 이름이 붙게 된다.
- ResNet은 기본적으로 **VGG-19**의 구조를 뼈대로 하고, 컨볼루션 층들을 추가해서 깊게 만든 후에, shortcut들을 추가하는 구조



Reference

"YOLOv11: An Overview of the Key Architectural Enhancements"(2024) Khanam et al

"End-to-End Chess Recognition" (2024) Masouris et al

Banerjee, N., Saha, D., Singh, A., and Sanyal, G. (2011). A simple autonomous robotic manipulator for playing chess against any opponent in real time. In Proceedings of the International Conference on Computational Vision and Robotics.

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., and Zagoruyko, S. (2020). End-to-end object detection with transformers. In European conference on computer vision, pages 213–229. Springer.

Chen, A. T.-Y., Kevin, I., and Wang, K. (2016). Computer vision based chess playing capabilities for the baxter humanoid robot. In 2016 2nd International Conference on Control, Automation and Robotics (ICCAR), pages 11–14. IEEE.

Chen, A. T.-Y. and Wang, K. I.-K. (2019). Robust computer vision chess analysis and interaction with a humanoid robot. Computers, 8(1):14.